

V2Ray / Xray Slow Speed on AWS EC2

Diagnosis & Fix Report — AWS EC2 (Singapore) vs Akamai/Linode VPS

Symptom	V2Ray/Xray tunnel on AWS EC2 t2.micro capped at ~10 Mbps download, while the same local ISP + package reached full ~100 Mbps through an Akamai (Linode) VPS.
Root Cause	Outdated Xray-core build + default Linux TCP congestion control (Cubic) on the EC2 instance. Not CPU, not the instance's network cap, not ISP routing.
Fix	Updated Xray-core via the X-UI panel and enabled Google's BBR congestion control, then rebooted. Speed became stable at 80–100 Mbps.

1. The Issue

Two VPN servers were set up using the same VLESS/V2Ray configuration approach, connected to through the same local ISP and the same data package, both hosted in the Singapore region:

- **Server A — AWS EC2 (t2.micro):** Download speed capped around 10 Mbps.
- **Server B — Akamai (Linode):** Download speed reached the full ~100 Mbps.

Since the ISP, package, and region were identical, the difference had to be caused by something specific to how each server/instance was configured or provisioned.

2. Investigation — What Was Ruled Out

2.1 CPU Credit Throttling (t2.micro)

t2.micro is a burstable instance type that relies on a CPU credit system. If credits run out, AWS throttles CPU to a low baseline. This was checked directly in CloudWatch during an active slow download:

- CPU Utilization: 3.77% peak (near idle)
- CPU Credit Usage: 0.34 (negligible)
- CPU Credit Balance: 289, flat — not draining

Conclusion: Not CPU credit throttling. Plenty of headroom available.

2.2 Instance Network Baseline Cap

A raw HTTP file download was tested directly against the EC2 instance's public IP (Apache serving a 500 MB test file), completely bypassing the V2Ray/Xray tunnel:

```
http://<ec2-public-ip>/test.bin
```

Result: 80–100 Mbps — full expected speed.

Conclusion: The instance's network interface, security group, and ISP→AWS routing path were all confirmed fine. The bottleneck exists specifically inside the V2Ray/Xray tunnel layer, not the raw network path.

2.3 ISP / Peering Routing

Since the direct HTTP test reached full speed on the same network path the tunnel uses, ISP routing and AWS Singapore peering were also ruled out as the cause.

3. Root Cause Identified

The server was running Xray-core as a subprocess managed by the **3X-UI panel** (`bin/xray-linux-amd64 -c bin/config.json`). Two issues were found stacked together, both invisible to standard CPU/network monitoring:

Issue 1 — Outdated Xray-core build

The panel had installed Xray-core once at setup time and never auto-updated it. Older Xray-core builds are known to be less efficient when handling VLESS + TCP + TLS traffic, adding processing overhead per connection. This overhead does not show up as high CPU usage in coarse-grained tools like AWS CloudWatch, but it throttles real throughput.

Issue 2 — Default TCP congestion control (Cubic)

Linux's default congestion control algorithm, Cubic, backs off bandwidth conservatively on paths with variable latency or occasional packet loss — common on ISP-to-AWS-Singapore routes depending on peering. Google's **BBR** algorithm is far more aggressive at reclaiming available bandwidth on exactly this kind of path.

Neither issue was visible in CPU or network graphs, which is why they had to be found by process of elimination rather than by reading a single metric.

4. The Fix — Step by Step

The following steps were performed using the 3X-UI panel management script on the EC2 instance. Run this on any fresh Xray/V2Ray server before troubleshooting anything else.

Step 1 — Open the X-UI management script

```
x-ui
```

Opens the 3X-UI panel management menu in the terminal.

Step 2 — Update Xray-core and the panel

```
Select option 2 (Update)
```

Updates Xray-core to the latest stable release. In this case it updated to version 26.7.28.

Step 3 — Enable BBR congestion control

```
Select option 26 (Enable BBR)
```

Switches the server's TCP congestion control from Cubic to BBR. Confirmed by checking:

```
net.core.default_qdisc = fq
net.ipv4.tcp_congestion_control = bbr
```

Step 4 — Restart Xray

```
Select option 15 (Restart Xray)
```

Applies the updated Xray-core binary cleanly.

Step 5 — Verify status

```
Select option 16 (Check Status)
```

Confirms: Panel state: Running, xray state: Running, no errors in the log output.

Step 6 — Reboot the VM

```
sudo reboot
```

Ensures the BBR/kernel-level changes and any pending kernel update are fully and cleanly applied.

Step 7 — Reconnect and retest

```
Reconnect the V2Ray/VLESS client and run a download speed test through the tunnel
```

Result: stable 80–100 Mbps, matching the Akamai server.

Result

After completing all steps, download speed through the V2Ray/VLESS tunnel on the AWS EC2 t2.micro instance became stable at 80–100 Mbps, matching the performance of the Akamai/Linode server.

5. Repeatable Checklist for Future Servers

Run this immediately after deploying any new EC2 (or similar cloud VPS) instance for V2Ray/Xray, before doing any other troubleshooting:

#	Action	Command / Menu Option
1	Update Xray-core via X-UI	x-ui → option 2 (Update)
2	Enable BBR congestion control	x-ui → option 26 (Enable BBR)
3	Restart Xray	x-ui → option 15 (Restart Xray)
4	Reboot the VM	sudo reboot
5	Reconnect client and test speed	Test through the actual tunnel, not a direct HTTP download

6. How to Diagnose This Yourself Next Time

- **Check CPU credits first** (only relevant on burstable instances like t2/t3): EC2 Console → Instance → Monitoring tab → CPU Credit Balance / Usage graphs.
- **Bypass the tunnel entirely** with a raw HTTP test: install a lightweight web server (e.g. Apache), serve a large dummy file, and download it directly by IP with the VPN off. This isolates network/instance issues from tunnel-software issues.

- **If the raw HTTP test is fast but the tunnel is slow**, the problem is in Xray/V2Ray itself — check the core version and TCP congestion control before anything else.
- **Always update Xray-core and enable BBR** as a standard step right after provisioning any new server for this purpose.

Security note: the original VLESS configuration link shared during this investigation contained a live UUID and server address. That UUID should be regenerated and reissued, since anyone with the string could connect through the server and consume bandwidth.